BÖLÜM 1 / MODULE 1

Konteyner Dünyası & Docker Temelleri

Container World & Docker Fundamentals

Yazılım dünyasının standart kargo konteynerleri: Sanal makinelerden Docker'a, multi-stage Spring Boot derlemelerine ve ağ yönetimine sıfırdan adım atış.

The standard shipping containers of software: From VMs to Docker, multi-stage Spring Boot builds, volumes, and networks from scratch.

1

BÖLÜM: KONTEYNER DÜNYASI & DOCKER TEMELLERI | MODULE: CONTAINER WORLD & DOCKER FUNDAMENTALS

"Ama Benim Bilgisayarımda Çalışıyordu!" – Sanal Makineler vs Konteynerler

"It Worked on My Machine!" – Virtual Machines vs Containers

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Eskiden bir tüccar kumaş, diğeri zeytinyağı fıçısı, öbürü gevşek buğday çuvalı taşırdı. Gemiyi yüklemek günler sürer, çuvallar patlar, her şey birbirine karışırdı! 1956'da standart çelik kargo konteyneri icat edildi. Artık gemi vincinin içinde ne olduğunu bilmesine gerek yok; standart kutuyu alır, koyar! Docker da yazılım dünyasının standart çelik kutusudur. Sanal Makine (VM) koca bir evi temeliyle taşımak gibidir; Docker Konteyneri ise sadece eşyalarınızı bavula koyup taşımak gibidir!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Back in the 1950s, dockers loaded odd wooden barrels, loose bags of grain, and fragile crates by hand. Cargo took days to load and frequently broke. In 1956, the standardized steel shipping container changed human trade forever! Cranes didn't care what was inside—only that it fit the standard locks. Docker is that exact shipping container for software. A Virtual Machine (VM) is like transporting an entire house including foundations; a Container is packing just your suitcase with only the essentials!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Sanal Makineler (Hypervisor Type 1/2) her uygulama için ayrı bir misafir işletim sistemi (Guest OS), ayrı sanal donanım ve kendi çekirdeğini (Kernel) ayağa kaldırır. Bu da gigabaytlarca RAM ve dakikalar süren açılış süresi demektir. Docker konteynerleri ise doğrudan ana makinenin Linux çekirdeğini paylaşır. İzolasyon iki temel Linux çekirdek özelliğiyle sağlanır: 1) Namespaces (Konteynerin sadece kendi süreçlerini, ağını ve dosya sistemini görmesini sağlar; sanal göz bağıdır), 2) Cgroups (Control Groups: CPU ve RAM kotasını sınırlar).

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Virtual Machines (Type 1/2 Hypervisors) virtualize physical hardware. Every VM runs a complete Guest OS kernel, consuming gigabytes of memory and taking minutes to boot. Docker containers share the host Linux kernel. Isolation is achieved via two core Linux kernel primitives: 1) Namespaces (isolate PID, Mount, Net, IPC, UTS, User so processes think they own the machine), and 2) Cgroups (Control Groups, metering and limiting CPU, RAM, and IOPS).

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Doğrudan sunucuda çalıştırmak (Kötü Yöntem):
# Java 17 mi kurulu, 21 mi? Hangi glibc sürümü var? Kimse bilmez!
java -jar target/spring-boot-k8s-eli5-1.0.0.jar

# Docker ile standart çelik konteynerde çalıştırmak (Modern Yöntem):
docker run -d -p 8080:8080 --name my-spring-coffee ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Konteynerler tam anlamıyla bir sanal makine değildir! Ana işletim sisteminin çekirdeğini (kernel) kullandıkları için, Linux tabanlı bir konteyner doğrudan saf Windows çekirdeğinde çalışmaz (arka planda WSL2 veya Hyper-V Linux VM gerekir).

🇬🇧 WATCH OUT

Containers are NOT lightweight virtual machines! Because they share the host kernel, Linux containers on Windows/macOS secretly run inside a thin Linux VM (WSL2 / Apple Hypervisor).

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Konteynerler Guest OS yükünü ortadan kaldırarak saniyeler içinde ayağa kalkar.
- Linux Namespaces görünürlüğü, Cgroups ise CPU/Bellek limitlerini kontrol eder.
- Java uygulaması her işletim sisteminde aynı JRE ve bağımlılıklarla garanti çalışır.

- Containers eliminate Guest OS overhead, starting in milliseconds.
- Namespaces provide visibility boundaries; Cgroups enforce resource limits.
- Guarantees identical Java runtime behavior across laptops, staging, and production.

2

BÖLÜM: KONTEYNER DÜNYASI & DOCKER TEMELLERI | MODULE: CONTAINER WORLD & DOCKER FUNDAMENTALS

Docker Mimarisi: İmaj (Image), Konteyner (Container) ve Registry

Docker Architecture: Images, Containers, and Registries

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Anneannenizin meşhur elmalı pasta tarifi defterde yazılıdır; buna 'İmaj' (Image) denir. Bu tariften fırında pişirdiğiniz, yiyebileceğiniz sıcacık pasta ise 'Konteyner'dir (Container). Tek bir tariften 100 tane özdeş pasta pişirebilirsiniz. Yemek tarifi siteleri (Docker Hub, GitHub Packages) ise 'Registry'dir!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Your grandma's apple pie recipe written in a cookbook is the 'Docker Image'. The actual warm, edible pie baked in the oven is the 'Container'. You can bake 100 identical pies from that single recipe. The online culinary recipe library is the 'Registry' (Docker Hub, Quay, GHCR)!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Docker Image, salt-okunur (read-only) katmanlardan (layers) oluşan bir OCI (Open Container Initiative) paketidir. Union File System (UnionFS/Overlay2) sayesinde her katman bir öncekinin üzerine bindirilir. Bir konteyner başlattığınızda, bu salt-okunur katmanların en tepesine incecik bir 'Yazılabilir Katman' (Writable Container Layer) eklenir. Dosya silindiğinde veya değiştirildiğinde 'Copy-on-Write' (CoW) mekanizması devreye girer.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

A Docker Image is an OCI-compliant package consisting of stacked read-only layers. Using an Overlay2 Union File System, each instruction in a Dockerfile generates an immutable layer. When a container runs, Docker attaches a thin writable layer on top. Any file modification uses the Copy-on-Write (CoW) strategy without mutating the underlying image.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# 1. Uzak depodan (Registry) imaj çek
docker pull eclipse-temurin:21-jre-alpine

# 2. İmajları listele
docker images

# 3. İmajdan çalışan bir konteyner üret (Tariften pasta pişir!)
docker run -d --name coffee-service -p 8080:8080 ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0

# 4. Çalışan konteynerleri denetle
docker ps
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Konteyneri durdurup sildiğinizde (docker rm), o en üstteki yazılabilir katmandaki tüm veriler (log-lar, geçici dosyalar) anında yok olur! Kalıcı veri için mutlaka 'Volume' kullanılmalıdır.

🇬🇧 WATCH OUT

When you destroy a container (`docker rm`), its writable top layer is destroyed forever! Never store production databases or uploads inside the container's ephemeral writable layer.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- İmaj değişmez (immutable) bir şablondur; Konteyner onun canlı örneğidir.
- Overlay2 dosya sistemi katmanları paylaşarak disk ve indirme tasarrufu sağlar.
- Registry'ler imajların güvenli dağıtıldığı global kütüphanelerdir.

- An Image is an immutable template; a Container is its running process instance.
- Overlay2 shares identical layers across images to save bandwidth and disk.
- Registries are centralized libraries for sharing versioned container images.

3

BÖLÜM: KONTEYNER DÜNYASI & DOCKER TEMELLERI | MODULE: CONTAINER WORLD & DOCKER FUNDAMENTALS

Java Spring Boot Uygulamasını Paketleme: Multi-Stage Build & Layered JAR

Packaging Java Spring Boot: Multi-Stage Builds & Layered JARs

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Kocaman un çuvallarını, mikseri ve kirli tencereleri salondaki yemek masasına koymazsınız! Mutfakta yemeği pişirir (Maven ile derleme), salona garsonla sadece şık porselen tabaktaki yemeği yollarsınız (JRE). Multi-stage build tam olarak budur: Derleme araçlarını çöpe at, sadece çalışan Java kodunu son imaja koy!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

You don't serve your dinner guests by wheeling 50kg sacks of flour and dirty mixing bowls into the dining room! You cook in the kitchen (Maven build) and carry only the clean porcelain plate to the table (JRE). Multi-stage builds discard the 800MB compiler tools and package only the lean 50MB runtime!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Spring Boot 2.3+ ile gelen 'Layered JAR' mimarisi, geleneksel tek parça (fat/uber) JAR'ları 4 mantıksal katmana böler: 1) dependencies (nadiren değişir), 2) spring-boot-loader, 3) snapshot-dependencies, 4) application (sizin kodunuz; her commit'te değişir). Dockerfile'da `java -Djarmode=layertools -jar app.jar extract` çalıştırarak bu katmanları ayırırız. Böylece 1 satır Java kodu değiştirdiğinizde Docker sadece 1MB'lık application katmanını yeniden derler; 150MB'lık kütüphaneleri baştan indirmez!

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Spring Boot layered JARs separate fat JARs into 4 distinct layers: 1) dependencies, 2) spring-boot-loader, 3) snapshot-dependencies, 4) application code. By running `java -Djarmode=layertools -jar app.jar extract`, Docker caches the 150MB third-party libraries indefinitely. When you change one Java class, Docker only invalidates and pushes the 2MB application layer!

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Dockerfile'da Katmanları Çıkarma ve Kopyalama:
FROM eclipse-temurin:21-jre-alpine AS extractor
WORKDIR /build
COPY target/spring-boot-k8s-eli5-1.0.0.jar app.jar
RUN java -Djarmode=layertools -jar app.jar extract

FROM eclipse-temurin:21-jre-alpine AS runner
WORKDIR /app
COPY --from=extractor /build/dependencies/ ./
COPY --from=extractor /build/spring-boot-loader/ ./
COPY --from=extractor /build/snapshot-dependencies/ ./
COPY --from=extractor /build/application/ ./
ENTRYPOINT ["java", "org.springframework.boot.loader.launch.JarLauncher"]
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Multi-stage kullanmazsanız, JDK derleyicisi ve Maven yerel deposu (/.m2) imajın içinde kalır; imaj boyutunuz 250MB yerine 1.2GB olur ve güvenlik açıkları tavan yapar!

🇬🇧 WATCH OUT

Without multi-stage builds, your JDK compiler, git, and /.m2 cache leak into production, inflating your image from 200MB to 1.5GB with huge security risks.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Multi-stage build ile derleme ortamı ve çalışma ortamı tamamen ayrılır.
- Spring Boot Layered JAR ile Docker derleme süreleri %90 hızlanır.
- Konteyner imajı küçüldükçe ağ transferi hızlanır, güvenlik açıkları azalır.

- Multi-stage builds decouple build tools from the minimal runtime image.
- Layered JAR caching reduces image build and push times by up to 90%.
- Smaller images transfer faster and significantly reduce attack surface.

4

BÖLÜM: KONTEYNER DÜNYASI & DOCKER TEMELLERI | MODULE: CONTAINER WORLD & DOCKER FUNDAMENTALS

Docker Veri Saklama ve Ağ: Volumes ve Bridge Networks

Docker Storage & Networking: Volumes and Bridge Networks

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Konteyner bir otel odası gibidir. Sabah check-out yaptığınızda temizlik görevlisi gelir ve odadaki her şeyi çöpe atar! Kalıcı eşyalarınızı otelin kiralık çelik kasasında (Volume) saklamalısınız. Odalar arası telefon hattı ise Docker Network'tür; oda numarasını (konteyner adını) çevirip doğrudan konuşursunuz.

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Containers are like luxury hotel rooms. When you check out, the maid throws everything in the room into the incinerator! To keep your laptop and passport safe, you store them in the hotel's secure basement locker (Docker Volume). The intercom phone between rooms is the Docker Network; just dial the room name to talk.

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Docker'da üç temel veri saklama yöntemi vardır: 1) Named Volumes (Docker tarafından `/var/lib/docker/volumes` altında yönetilir; en yüksek performans ve taşınabilirlik), 2) Bind Mounts (Host makinedeki doğrudan bir klasörü `/Users/.../data` konteyner içine bağlar; geliştirme anında kod senkronizasyonu için idealdir), 3) tmpfs (Sadece RAM'de saklanır). Ağ tarafında ise varsayılan `bridge` ağı, konteynerler arasında yerleşik DNS çözünürlüğü sunar. Konteynerler birbirine IP ile değil isimleriyle (`postgres-db:5432`) erişir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Storage strategies: 1) Named Volumes (managed under `/var/lib/docker/volumes`, isolated from host OS quirks, best performance), 2) Bind Mounts (mounts arbitrary host paths, ideal for local dev live-reload), 3) tmpfs (stored strictly in host memory). On the networking side, user-defined bridge networks provide automatic embedded DNS resolution between containers by container name.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# 1. Özel bir bridge ağı oluştur
docker network create coffee-net

# 2. Kalıcı bir veri diski (Volume) oluştur
docker volume create postgres-data

# 3. PostgreSQL'i ağa ve diske bağlayarak başlat
docker run -d --name postgres-db --network coffee-net \
  -v postgres-data:/var/lib/postgresql/data \
  -e POSTGRES_PASSWORD=secret -e POSTGRES_DB=coffeedb postgres:16-alpine

# 4. Spring Boot'u aynı ağa bağla (IP yerine doğrudan 'postgres-db' adını kullanır!)
docker run -d --name coffee-api --network coffee-net -p 8080:8080 \
  -e SPRING_DATASOURCE_URL=jdbc:postgresql://postgres-db:5432/coffeedb \
  ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Varsayılan (default) bridge ağında konteyner isimleriyle otomatik DNS çözümlemesi çalışmaz! İsimle haberleşme için mutlaka `docker network create` ile kendi özel ağınızı oluşturmalısınız.

🇬🇧 WATCH OUT

The default legacy docker0 bridge does NOT provide automatic DNS resolution by container name! Always create a user-defined bridge network.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Named Volumes konteyner silinse bile veritabanı verilerini korur.
- Named volumes persist state even when containers are stopped and destroyed.
- Docker yerel DNS'i sayesinde konteynerler IP değişse bile birbirini bulur.
- Docker embedded DNS resolves container names dynamically without static IPs.
- Veritabanı ve Spring Boot uygulamalarını aynı özel ağa bağlamak şarttır.
- Co-locating apps and databases on dedicated networks isolates traffic securely.

5

BÖLÜM: KONTEYNER DÜNYASI & DOCKER TEMELLERI | MODULE: CONTAINER WORLD & DOCKER FUNDAMENTALS

Docker Compose ile Çoklu Servis: Spring Boot + PostgreSQL Çiftliği

Multi-Service Stacks with Docker Compose: Spring Boot + PostgreSQL

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Restoranda tek başına yemek yapmak kolaydır. Ama aynı anda ekmek fırınını, et ızgarasını ve tatlı reyonunu senkronize başlatmak için bir şefe ve nota kağıdına ihtiyacınız vardır. Docker Compose o nota kağıdıdır (`docker-compose.yml`). Tek bir tuşla (`docker compose up`) tüm orkestra kusursuz sırada sahneye çıkar!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Playing a solo flute is easy. Conducting a 20-piece symphony requires sheet music! Docker Compose is the master sheet music (`docker-compose.yml`). With a single command (`docker compose up`), your database, Redis cache, and Spring Boot backend spawn in perfect harmony!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Docker Compose bildirici (declarative) bir YAML dosyasıyla servisleri, ağları ve diskleri tek bir çatı altında tanımlar. `depends_on` mekanizması başlatma sırasını belirler; `healthcheck` bloğu ile birleştirildiğinde Spring Boot servisinin PostgreSQL tamamen 'ready' olana kadar beklemesi sağlanabilir. Ortam değişkenleri (`environment`), port haritalamaları (`ports`) ve secret dosyaları tek merkezden yönetilir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Docker Compose declaratively defines multi-container applications. Using `depends_on` with `condition: service_healthy`, Compose ensures your Spring Boot service pauses booting until PostgreSQL is genuinely accepting TCP connections, eliminating startup connection crashes.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```yaml
# docker-compose.yml
services:
  postgres-db:
    image: postgres:16-alpine
    container_name: postgres-db
    environment:
      POSTGRES_DB: coffeedb
      POSTGRES_USER: coffeeuser
      POSTGRES_PASSWORD: secretpassword
    volumes:
      - postgres-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U coffeeuser -d coffeedb"]
      interval: 5s
      timeout: 5s
      retries: 5

  coffee-app:
    build: .
    container_name: coffee-app
    ports:
      - "8080:8080"
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres-db:5432/coffeedb
      SPRING_DATASOURCE_USERNAME: coffeeuser
      SPRING_DATASOURCE_PASSWORD: secretpassword
    depends_on:
      postgres-db:
        condition: service_healthy

volumes:
  postgres-data:
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

`depends_on` tek başına sadece veritabanı konteynerinin başladığını doğrular; veritabanı motorunun bağlantı kabul etmeye hazır olduğunu garanti etmez. Mutlaka `condition: service_healthy` ile birlikte kullanılmalıdır!

🇬🇧 WATCH OUT

A bare `depends_on` only waits for the container process to spawn, NOT for PostgreSQL to be ready! Always pair it with healthcheck conditions.

🎯 Özet Çıkarımlar

• Tek bir komut (`docker compose up -d`) ile tüm mikroservis yığını ayağa kalkar.

• Healthcheck blokları servisler arası yarış durumlarını (race condition) önler.

• Yerel geliştirme ortamını üretim ortamına en yakın hale getirir.

🎯 Key Takeaways

• Spawn entire multi-tier architectures with a single `docker compose up -d` command.

• Healthchecks eliminate startup race conditions between apps and databases.

• Mirrors production topology on local developer machines seamlessly.

6

BÖLÜM: KONTEYNER DÜNYASI & DOCKER TEMELLERI | MODULE: CONTAINER WORLD & DOCKER FUNDAMENTALS

Container Güvenliği & İmaj Optimizasyonu: Distroless, Alpine & Non-Root

Container Security & Image Optimization: Distroless, Alpine & Non-Root

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Uçağa binerken yanınıza 150 kiloluk gardırop almazsınız; hem fazla para ödersiniz hem de hırsızlara davetiye çıkarırsınız! Standart Ubuntu imajında hacker'ların kullanabileceği 500 tane gereksiz araç (curl, apt, bash) vardır. Zırhlı mini imajda (Alpine veya Distroless) ise sadece Java çalışır. Üstüne bir de kapıya 'ROOT GİREMEZ' tabelası asarız!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

You don't travel with a 150kg solid oak wardrobe on an airplane! Bloated images carry 500 unused utilities (bash, curl, package managers) that attackers love to exploit. Hardened minimal images (Alpine, Distroless) contain ONLY your Java binary. And we lock out the superuser by hanging a strict 'NO ROOT ALLOWED' sign!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Varsayılan olarak Docker konteynerleri `root` (UID 0) kullanıcısıyla çalışır. Konteynerden kaçış (container breakout) açığı oluşursa, saldırgan ana sunucuda da tam yetkili root olur! Bu yüzden: 1) Dockerfile içinde özel bir `appuser` (UID 10001) tanımlanmalıdır. 2) Paket yöneticisi ve kabuk (shell) bile içermeyen Google Distroless veya hafif Alpine imajları tercih edilmelidir. 3) Trivy veya Grype ile imajdaki CVE güvenlik açıkları CI pipeline'ında taranmalıdır.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

By default, containers execute as `root` (UID 0). In the event of a Linux kernel or container breakout vulnerability, an attacker gains root on the physical host. Mitigation requires: 1) Enforcing non-root users (`USER 10001`), 2) Using shell-less Distroless bases, 3) Automated scanning for known CVEs via Trivy.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# Güvenli Dockerfile Snippet:
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app

# 1. Root olmayan grup ve kullanıcı yarat
RUN addgroup -S appgroup -g 10001 && \
    adduser -S appuser -u 10001 -G appgroup

# 2. Sahipliği ver ve kullanıcıya geç
COPY --chown=appuser:appgroup target/app.jar app.jar
USER 10001:10001

# 3. Bellek sınırını JVM'e bildir (K8s pod çökmesini engeller)
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75.0 -XX:+UseG1GC"
ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar app.jar"]
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Konteyneri root olmayan kullanıcıyla çalıştırırken port numarasını 1024'ün altında (örneğin port 80) seçmeyin; Linux geleneksel olarak 1024 altındaki portları sadece root'a tahsis eder. Her zaman 8080 gibi yüksek portlar seçin!

🇬🇧 WATCH OUT

Non-root users cannot bind to privileged ports below 1024 (e.g. port 80). Always configure Spring Boot on high ports like 8080.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- Üretim ortamında ASLA konteynerleri root kullanıcısıyla çalıştırmayın.
- Distroless ve Alpine taban imajları saldırı yüzeyini ve dosya boyutunu küçültür.
- JVM `-XX:MaxRAMPercentage` ayarı bellek taşmalarını (OOMKilled) önler.

- NEVER execute containers as root in enterprise environments.
- Distroless and Alpine minimize the attack surface and download size.
- JVM container flags align Java heap limits with container cgroup limits.